

Scalability: Vertical and Horizontal Scaling

Learn: Scalability — Why need to Scale?

Il perché

Il problema ereditato dalla Lezione 3: **una singola istanza dell'API Service si sta sovraccaricando** a causa dell'aumento del traffico.

*"La scalabilità è un concetto fondamentale e **non è più solo un optional**. È un requisito non funzionale per qualsiasi applicazione e-commerce di successo che punti a centinaia di migliaia o milioni di potenziali utenti."*

Definizione

La scalabilità è la capacità di un sistema, di una rete o di un processo di gestire un carico di lavoro crescente — ovvero la sua capacità di espandersi per adattarsi a tale crescita.

Come si misura nelle applicazioni web:

Il **numero di richieste o utenti simultanei** che il sistema può supportare efficacemente **senza un degrado significativo delle prestazioni** (aumento di latenza, instabilità).

Definizione: limite di scalabilità (scalability limit) — il punto in cui l'applicazione **non è più in grado di gestire efficientemente un numero maggiore di richieste simultanee**.

L'obiettivo dello scaling

Prevenire i downtime, ridurre la latenza e supportare la crescita del business. Concretamente significa **aggiungere o potenziare risorse di calcolo**: CPU, memoria, larghezza di banda di rete, o l'intero hardware.

Le due strategie

Strategia	In una frase
Scalabilità verticale	Potenziare il server esistente
Scalabilità orizzontale	Aggiungere altro hardware al pool di risorse

Vertical and Horizontal Scaling

Vertical and Horizontal Scaling"

A) Scalabilità verticale (Vertical Scaling / "Scale Up")

Definizione: potenziare la macchina esistente su cui gira l'applicazione.

Cosa si fa concretamente:

- aggiungere **core CPU**
- aumentare la **RAM**
- passare a **SSD più veloci**
- migliorare l'**interfaccia di rete**

I 2 vantaggi

1. Semplicità di implementazione

*"Il codice esistente **non necessita necessariamente di modifiche**. Funziona semplicemente su una macchina più potente."*

Questo è il grande pregio: è una soluzione **infrastrutturale**, non architetturale. Non tocchi il software.

2. Coerenza dei dati più semplice

Per applicazioni o database che girano su **un singolo nodo**, mantenere la coerenza è molto più semplice (non c'è nulla da sincronizzare tra macchine).

Le 3 limitazioni

1. Limiti fisici e di costo ★

*"Esiste un **limite fisico** alla potenza che una singola macchina può raggiungere. Oltre un certo punto, **il costo dell'hardware server di fascia alta aumenta esponenzialmente**. Si arriva a un punto in cui non è più possibile — o non è più economicamente sostenibile — usare macchine più potenti."*

La lezione descrive due curve:

	Scenario ideale	Realtà
Aspettativa	Più risorse → più prestazioni, linearmente —	
Realtà	—	A un certo punto, anche aggiungendo risorse le prestazioni non migliorano più . Si raggiunge il limite di scalabilità.

Perché la curva si appiattisce: contese sui lock, saturazione della banda di memoria, limiti del kernel sui file descriptor, contention sul disco. Aggiungere la 64^a CPU non raddoppia il throughput della 32^a.

2. Single Point of Failure ★

*"Hai ancora **un solo server**, per quanto estremamente potente. Se quel server si guasta — problema hardware, crash del sistema operativo — **l'applicazione si blocca completamente**."*

3. Downtime per gli aggiornamenti

Un upgrade fisico dell'hardware richiede spesso di **spegnere la macchina**, causando downtime — *"generalmente insostenibile"* per un e-commerce che deve stare su 24/7.

Quando basta

*"Se la nostra applicazione deve gestire **da 2.000 a 20.000 utenti simultanei**, lo scaling verticale potrebbe essere una soluzione. Ma se raggiungiamo **milioni di richieste**, un singolo server — per quanto potente — non sarà sufficiente."*

B) Scalabilità orizzontale (Horizontal Scaling / "Scale Out")

Definizione: distribuire il carico dell'applicazione **su più server** anziché su un singolo server potente.

*"Si potrebbero avere due, tre, dieci o persino **centinaia di server meno potenti ma più convenienti** che lavorano insieme. È sufficiente **aggiungere ulteriori istanze dell'applicazione**."*

I 4 vantaggi

1. Scalabilità complessiva molto più alta ★

*"È possibile raggiungere livelli di scalabilità **molto più elevati** aggiungendo hardware al pool. **Spesso è più semplice aggiungere server standard che rendere un singolo server infinitamente potente**."*

Teoricamente non ha un limite superiore rigido: aggiungi nodi.

2. Tolleranza ai guasti e alta disponibilità ★

*"Se un'istanza si guasta, **gli altri server possono continuare a gestire il traffico**, soprattutto usando un load balancer. Questo garantisce **ridondanza**."*

Questo risolve direttamente il problema del single point of failure dello scaling verticale.

3. Convenienza economica

Un pool di hardware standard (commodity) meno costoso può risultare **più economico** di un singolo server altamente sofisticato.

4. È lo standard moderno

*"È il metodo preferito per scalare le **architetture distribuite e cloud-native**."*

Le 3 complessità

1. Complessità architetturale — l'applicazione deve essere progettata per funzionare in un ambiente distribuito.

2. Gestione dello stato ★ — la sfida centrale.

*"Se un'istanza dell'applicazione ha **stato**, sincronizzare o gestire quello stato tra più istanze diventa una sfida significativa. **Ecco perché le applicazioni stateless sono altamente desiderabili per la scalabilità orizzontale**."*

Definizione: applicazione stateless — un'applicazione che **non conserva in memoria alcuna informazione tra una richiesta e l'altra**. Ogni richiesta contiene tutto il necessario per essere processata. Qualsiasi stato vive **esternamente**.

Il problema concreto: se l'utente fa login sull'istanza A e la sua sessione è salvata nella RAM di A, alla richiesta successiva il load balancer potrebbe mandarlo sull'istanza B — che non sa chi è. Risultato: logout casuali.

Le due soluzioni:

Soluzione	Come funziona	Giudizio
Stato esternalizzato	Sessione salvata in una cache distribuita (Redis) o in un database, accessibile da tutte le istanze	✅ Preferita
Session affinity (sticky sessions)	Il load balancer manda sempre lo stesso utente sulla stessa istanza	⚠️ Fragile: se quell'istanza cade, l'utente perde tutto; e il carico si sbilancia

Nota come questo giustifichi retroattivamente la scelta di **Redis** nella Lezione 3: era già una preparazione a questo momento.

3. Serve un load balancer — un componente **davanti** alle istanze per distribuire uniformemente le richieste.

Nota aggiuntiva: se le istanze devono **comunicare tra loro**, si introducono latenza di rete e complessità.

Confronto e approccio ibrido

	Verticale (Scale Up)	Orizzontale (Scale Out)
Cosa fai	Potenzi la macchina	Aggiungi macchine
Modifiche al codice	Nessuna	L'app deve essere stateless
Limite	Fisico ed economico	Praticamente nessuno
Fault tolerance	❌ Single point of failure	✅ Ridondanza
Downtime per upgrade	❌ Sì	✅ No (aggiungi/togli a caldo)
Componenti extra	Nessuno	Load balancer
Adatto a	Applicazioni di dimensioni moderate, semplicità	Alta scalabilità, fault tolerance, ambienti cloud

L'approccio ibrido (la realtà)

*"In molti scenari reali si usa un **approccio ibrido**. Si scala verticalmente ogni singola istanza fino alla **dimensione ottimale in termini di costo/beneficio**, e poi si scala orizzontalmente aggiungendo altre istanze di quella dimensione."*

In pratica: non usi macchine minuscole (overhead sproporzionato) né mostri da 2 TB di RAM (costo esponenziale). Trovi il "sweet spot" e poi ne metti N.

What is a Load Balancer? Use Load Balancer Splitting the Load

Definizione

Un load balancer è un dispositivo hardware o un'applicazione software dedicata che funge da "controllore del traffico" per i server backend.

Come si posiziona: davanti alle istanze del servizio. Accetta il traffico di rete e applicativo in entrata (le richieste API della SPA) e lo **distribuisce intelligentemente** tra le istanze disponibili.

I 3 obiettivi principali

1. **Migliorare la reattività** — garantendo che nessun singolo server diventi un collo di bottiglia
2. **Aumentare disponibilità e fault tolerance** — instradando il traffico **lontano dai server guasti**
3. **Migliorare l'utilizzo delle risorse** — distribuendo il carico uniformemente

Le 4 funzioni chiave

1. Distribuzione del carico

Distribuisce le richieste tra un gruppo di server, **impedendo il sovraccarico di uno solo**.

2. Abilitare l'elasticità dello scaling orizzontale ★

*"Quando il traffico aumenta, è sufficiente **aggiungere altre istanze al pool** e il load balancer inizierà automaticamente a distribuire traffico verso di esse. Al contrario, **si possono rimuovere istanze nei periodi di basso traffico**."*

È questo che rende lo scaling orizzontale realmente pratico ed economico: paghi solo ciò che usi.

3. Health check ★

*"La maggior parte dei load balancer esegue **controlli periodici sullo stato di salute** dei server backend. Se un'istanza **non risponde o non supera il controllo**, il load balancer **smette di inviarle traffico** e lo reindirizza alle istanze funzionanti."*

È il meccanismo che trasforma la ridondanza teorica in **alta disponibilità reale**.

4. Session persistence (session affinity)

In alcune applicazioni è essenziale che tutte le richieste di una sessione utente vadano allo **stesso server backend**. I load balancer possono essere configurati per questo — *"sebbene sia spesso meno critico per le API moderne, che sono stateless"*.

Funzione aggiuntiva: SSL Termination

Alcuni load balancer gestiscono la **decrittazione del traffico HTTPS**, alleggerendo i server applicativi da un lavoro che richiede molta CPU.

I 4 algoritmi di bilanciamento

*"I load balancer **non instradano il traffico in modo casuale**: usano algoritmi specifici per determinare quale server debba gestire ogni richiesta."*

Algoritmo	Come funziona	Quando usarlo
Round Robin	Distribuisce le richieste in ordine sequenziale e ciclico : server 1, server 2, server 3, server 1...	Il più semplice. Va bene se i server sono identici e le richieste omogenee
Least Connections	Invia la nuova richiesta al server che ha attualmente il minor numero di connessioni attive	★ Molto efficace quando i tempi di elaborazione variano significativamente tra richieste
IP Hash	Esegue l' hash dell'IP del client per scegliere il server. Lo stesso client va sempre sullo stesso server	Utile per mantenere lo stato di sessione senza sticky session esplicite
Weighted (ponderato)	Si assegna un peso a ogni server; il traffico viene distribuito proporzionalmente ai pesi	Quando i server hanno capacità diverse (alcuni più potenti)

★ Consistent Hashing (Hashing Coerente)

Questa è la parte tecnicamente più interessante della lezione.

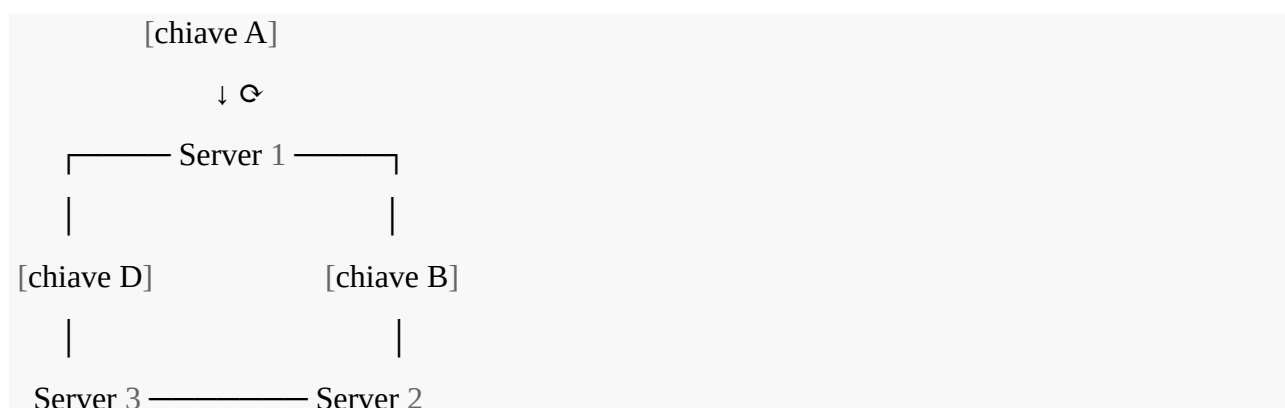
Definizione:

Un algoritmo avanzato, ideale per ambienti dinamici, che minimizza la riassegnazione di chiavi/richieste quando il pool di server cambia.

Quando serve: nei sistemi distribuiti che gestiscono **dati in cache** o **connessioni con stato**.

Il meccanismo dell'anello

1. Sia i server sia le chiavi dei dati vengono mappati su un **anello concettuale** (un cerchio di valori hash)
2. Per determinare **quale server gestisce una certa chiave**: si fa l'hash della chiave per trovarne la posizione sull'anello, poi ci si **muove in senso orario** fino a incontrare il primo server. **Quel server è responsabile di quella chiave.**



La "magia": cosa succede quando cambia il pool

"Se si aggiunge un server, questo viene posizionato sull'anello. Solo le chiavi che si trovano nel segmento immediatamente precedente al nuovo server devono essere riassegnate a quest'ultimo."

"Se si rimuove un server, le chiavi che gestiva vengono semplicemente riassegnate al server successivo nell'anello."

"In entrambi i casi, è necessario riassegnare solo una piccola parte delle chiavi, non l'intero set."

Perché è così importante — il confronto

Per capire il valore, confrontiamolo con l'hashing banale ($\text{server} = \text{hash}(\text{key}) \% N$):

	Hash modulo N	Consistent Hashing
Aggiungi 1 server a un pool di 4	% 4 diventa % 5 → quasi tutte le chiavi cambiano server	~1/5 delle chiavi viene riassegnato
Effetto sulla cache	Cache completamente invalidata → tempesta di richieste al database	Perdita di cache minima
	<i>"Riduce drasticamente la perdita di cache (cache miss) quando i server di cache vengono scalati."</i>	

Questo è il motivo per cui Redis Cluster, Memcached distribuiti, DynamoDB e Cassandra lo usano tutti.

I nodi virtuali

*"Si può ottenere una **migliore distribuzione del carico** usando tecniche come i **nodi virtuali (virtual nodes)** o la **replica**, in cui **ogni server fisico è rappresentato più volte sull'anello**."*

Perché serve: con pochi server, la distribuzione sull'anello può essere sbilanciata (un server prende un arco molto più grande). Rappresentando ogni server fisico come 100-200 punti sull'anello, la distribuzione diventa statisticamente uniforme.

Conclusione: *"Questa è l'opzione migliore quando si lavora con sistemi distribuiti in cui il numero di nodi è dinamico."*

Design: E-Shop App with Horizontal Scaling and Load Balancer

Toolkit aggiornato

Requisiti funzionali: invariati.

Nuovi requisiti non funzionali:

1. **Alta scalabilità** — l'API Service deve gestire un numero elevato e **variabile** di richieste, distribuendo efficacemente il carico
2. **Alta disponibilità a livello applicativo** — il sistema deve continuare a funzionare **anche se una o più istanze dell'API Service falliscono**

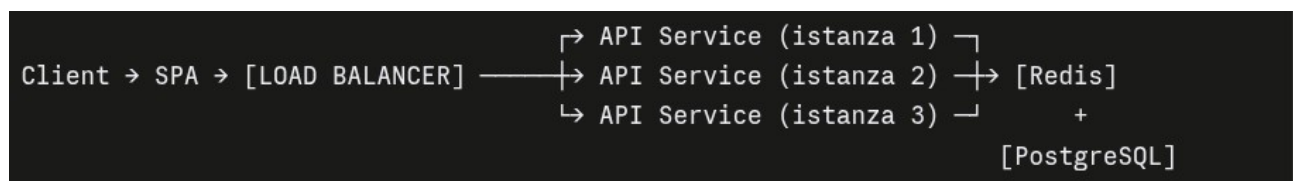
Nuovi vincoli/assunzioni — due, entrambi importanti:

1. L'**API Service** deve essere **progettato stateless**, o con lo stato gestito esternamente (Redis o database), per consentire uno scaling orizzontale efficiente
2. La **configurazione e gestione del load balancer** diventa una nuova considerazione operativa

Nuovi elementi nel toolkit:

- **Horizontal scaling** come strategia di deployment dell'API Service
- **Load balancer** per distribuire il carico

Il design



Cosa cambia rispetto alla Lezione 3:

1. Le richieste della SPA **passano prima attraverso il load balancer**
2. L'**application tier non è più una singola istanza**: ci sono **N istanze identiche** dell'API Service modulare (la slide ne mostra 3, "ma potrebbero essere 2, 10 o più a seconda del carico")

Cosa contiene ogni istanza:

"Ogni istanza API è una copia completa della logica dell'applicazione backend, inclusi il modulo Catalog, il modulo Basket, il modulo Ordering, ecc."

Questo è il punto: **si replica tutto**, anche i moduli che non hanno bisogno di scalare. È il limite che porterà ai microservizi.

Le risorse condivise:

- ogni istanza si connette alla **cache Redis condivisa**
- ogni istanza si connette al **database PostgreSQL condiviso** per la persistenza

Nota architetturale importante: il load balancer risolve il collo di bottiglia sull'**application tier**, ma Redis e PostgreSQL restano **condivisi e singoli**. Sono i prossimi colli di bottiglia in agguato — e infatti la **Lezione 10** si occuperà proprio dello scaling del database.

Nota aggiuntiva della lezione: lo stesso schema si può applicare **anche al presentation tier** — un load balancer davanti a più istanze della WebApp.

Scelte tecnologiche

Componente	Opzioni
Presentation	Blazor (invariato)
Load balancer software	nginx, HAProxy
Load balancer gestito	AWS Elastic Load Balancer, Azure Load Balancer, Google Cloud

Componente	Opzioni
(cloud)	Load Balancer
Algoritmo	Round Robin, Least Connections, o Consistent Hashing
Application	Più istanze del progetto ASP.NET Core modulare
Data	PostgreSQL

★ **Il requisito non negoziabile: stateless**

*"Un aspetto **fondamentale** in fase di progettazione: per un bilanciamento del carico semplice tra queste istanze, l'API Service deve idealmente essere stateless."*

*"Questo significa che **qualsiasi dato relativo a una sessione utente o a un'interazione multi-richiesta NON deve essere memorizzato nella memoria di una specifica istanza**. Se serve stato, va memorizzato esternamente — nella cache distribuita Redis o nel database PostgreSQL — **accessibile a tutte le istanze**."*

Code: E-Shop App with Horizontal Scaling

Si riusa la stessa soluzione della Lezione 3 (5-eshop-modular-monoliths-add-cache), con **una sola modifica**.

★ **La modifica: una riga**

In AppHost/Program.cs:

```
var apiService = builder.AddProject<ApiService>("apiservice")
    .WithReference(db)
    .WithReference(cache)
    .WithReplicas(3); // ← L'UNICA AGGIUNTA
```

*"L'unica modifica per abilitare la scalabilità orizzontale in locale è l'aggiunta della funzione **WithReplicas**. Questo semplice comando dice ad Aspire di avviare e gestire **tre istanze (repliche)** del progetto API Service."*

Cosa fa Aspire automaticamente

Quando l'applicazione viene eseguita, Aspire si occupa di:

1. **Assegnare porte diverse a ciascuna replica**
2. Fornire **service discovery integrato**
3. Fornire un **certo grado di load balancing**

*"Quando altri servizi — come la nostra WebApp — tentano di comunicare con l'API Service, la WebApp **continua a inviare richieste al nome logico del servizio (apiservice)**, e Aspire **le instrada a una delle repliche disponibili**."*

Questo è il punto elegante: **il codice della WebApp non cambia di una riga**. Continua a chiamare `https+http://apiservice`, e l'infrastruttura fa il resto. È esattamente il comportamento che un load balancer + service discovery producono in produzione.

Il resto del codice — backing services PostgreSQL e Redis, moduli, endpoint — **rimane invariato**.

Cosa si osserva nella dashboard

*"Invece di un singolo API Service, ora vediamo **tre istanze dell'API Service**, ognuna con un **colore e un nome diversi**."*

I nomi sono generati con suffissi casuali (es. `apiservice-acb...`, `apiservice-bbq...`, `apiservice-k...`), e i **colori** sono la chiave per leggere le tracce dopo.

Nel resource graph:

*"La WebApp invia richieste a **tre istanze dell'API Service**. **Ogni replica è a sua volta connessa al database PostgreSQL condiviso e alla cache condivisa**."*

Il test del carico: il progetto RequestSender

Il problema del test:

*"Il test manuale tramite l'interfaccia utente **non ci mostrerebbe facilmente la distribuzione del carico**."*

Serve generare traffico in volume. Per questo esiste il progetto console **RequestSender** (lo trovi nella cartella `5-eshop-modular-monoliths-add-cache/RequestSender`).

Cosa fa il codice:

1. Crea un **HttpClient**
2. Imposta la **BaseAddress** sull'URL dell'API Service
3. Invia **10 richieste HTTP GET** all'endpoint `/products`

⚠ Il dettaglio pratico importante sull'URL:

*"Normalmente, lavorando **dentro** un progetto Aspire, si può usare il nome logico `https+http://apiservice` — come fa la WebApp — perché **il service discovery di Aspire lo risolve**. Ma qui ci troviamo in un **progetto esterno** all'ambiente Aspire. Per questo **serve un URL reale**."*

Quindi bisogna:

1. Aprire la dashboard Aspire
2. Trovare l'endpoint esposto dell'API Service (nell'esempio: `https://localhost:7029`)
3. **Copiarlo e incollarlo** nel RequestSender

Questo è un dettaglio che vale la pena capire: **il service discovery funziona solo dentro la rete logica dell'orchestratore**. Chi sta fuori ha bisogno di un indirizzo concreto — esattamente il ruolo che in produzione avrà l'**API Gateway** (Lezione 8).

★ Il risultato: la prova visiva nel tracing

Si esegue RequestSender, che invia 10 richieste. Poi si apre la sezione **Traces** della dashboard:

*"Si vedono **dieci richieste** al progetto API Service. **I diversi colori rappresentano le diverse repliche**. Abbiamo inviato **quattro richieste** all'istanza ACB, **una a un'altra**, e **quattro alla replica BBQ**."*

*"Con tre repliche, abbiamo inviato dieci richieste, e questo è chiaramente visibile dalle icone colorate nella visualizzazione della traccia. **Questa visualizzazione conferma che il carico è stato distribuito correttamente.**"*

Il concetto dimostrato: la stessa identica richiesta (GET /products) viene servita da **istanze diverse**, in modo trasparente per il chiamante. È lo scaling orizzontale in azione.

Problem: Ship New Features Faster to Compete the Market

EVAL di questa iterazione → **PROBLEM** della Lezione 5. È il momento in cui i microservizi diventano necessari.

Lo scenario

L'app ha grande successo ed è un attore importante sul mercato. Ma il successo porta **pressione a innovare costantemente e rilasciare velocemente**:

- il **team marketing** vuole un nuovo motore di raccomandazione
- il **team vendite** ha bisogno di strumenti promozionali avanzati
- il **team pagamenti** deve integrare **tre nuovi gateway di pagamento**

Tutto **il più rapidamente possibile**.

Nel frattempo il team di sviluppo si è ampliato in **team specializzati**: catalogo prodotti, ordini, pagamenti, identità. Ciascuno vuole rilasciare **in modo rapido e indipendente**.

L'osservazione sui pattern di carico ★

*"Notiamo che **diverse parti del nostro API Service presentano pattern di carico significativamente diversi**. Il catalogo prodotti, soprattutto con la cache, gestisce efficientemente un traffico di **lettura elevato**; ma i componenti di elaborazione degli ordini hanno **esigenze di scalabilità specifiche durante i picchi di vendita**, diverse da quelle del resto dell'applicazione."*

Questo è il punto tecnico centrale: **profili di carico eterogenei dentro un'unica unità di deployment**.

Il problema definito

*"Nonostante la modularità interna e la scalabilità orizzontale, **l'API Service rimane un'unica grande codebase distribuita come un'unica unità**. Questa natura monolitica a livello di deployment diventa la radice del nuovo problema: **la monoliticità rallenta l'innovazione, frena i team e limita la scalabilità indipendente.**"*

La frase da cui parte tutto il resto del corso: **la modularità ha risolto l'organizzazione del codice, ma NON il deployment**.

Le 5 sfide

1. Velocità di sviluppo ridotta, time-to-market lento ★

- **Tutti i team sono vincolati alla stessa codebase**

- Modifiche di un team possono impattare gli altri → servono **test di regressione completi dell'intera applicazione per ogni modifica**
- **Una pipeline di deployment unica per tutti**
 - *"Una funzionalità urgente può essere bloccata dalla lentezza di un altro team"*

2. Autonomia dei team ridotta, coordinamento eccessivo

I team **non possono scegliere autonomamente** gli strumenti migliori né la propria frequenza di rilascio. Alto rischio di **conflitti di integrazione e dipendenze incrociate**.

3. Scalabilità rigida ★

"Tutti i moduli vengono scalati insieme, anche se serve più capacità a un solo modulo. Ad esempio, il catalogo prodotti non può essere scalato diversamente dal modulo di elaborazione ordini, anche se hanno requisiti di risorse differenti."

Questo è lo spreco concreto: se per il Black Friday ti servono 20 istanze di Catalog, ottieni anche 20 istanze di Payment, Identity, Shipping — che stanno lì a consumare RAM.

4. Rigidità dello stack tecnologico

"È difficile per un team adottare un nuovo linguaggio o framework ideale per le proprie esigenze specifiche senza impattare l'intera applicazione."

Il team Recommendation vorrebbe Python per il machine learning. Non può.

5. Isolamento dei guasti assente ★

"Un guasto critico o una memory leak in un modulo può compromettere la stabilità dell'intera applicazione e impattare tutti gli altri moduli in esecuzione in quel processo."

Questo è forse il punto più severo: **condividere un processo significa condividere i crash**. Un bug nel modulo Shipping può far cadere anche il Catalog.

L'esigenza fondamentale

"L'esigenza fondamentale è una vera indipendenza per team, funzionalità e scalabilità."

Le tre necessità concrete:

Necessità	Dettaglio
Autonomia dei team	Team specializzati devono poter sviluppare, testare, distribuire e scalare le proprie funzionalità in modo veramente indipendente
Rilascio più veloce	Le modifiche a una funzionalità non devono richiedere retest e redeploy dell'intero backend
Ottimizzazione di risorse e costi	Scalare singole parti — solo il servizio Catalog, solo il servizio Payment — in base alle loro esigenze specifiche
La conclusione	

*"Questo livello di autonomia in sviluppo, deployment e scalabilità è **molto difficile da raggiungere con un'architettura monolitica, anche modulare**. Di fronte a queste esigenze, **esploreremo le architetture a microservizi come soluzione**."*

